JPA, Part 1

Object-relational mapping.
JPA/Hibernate in a single-table database.

Learning goals

1. Get an overview of object-relational mapping (ORM).
2. Learn to carry out single-table ORM with JPA and Hibernate.

Directions of development

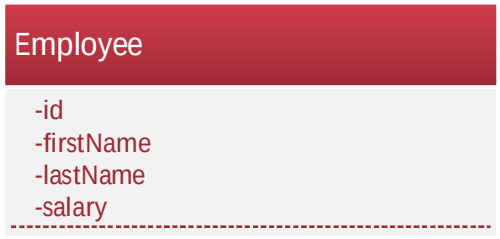
- JPA supports two directions of development:

1. Database first
   - Database schema designed in advance.
   - The tables and indices are generated outside the application.
   - The data can pre-exist.
   - This is a typical approach for most information system development projects.
   - The Java classes can (at least technically) be auto-generated from the database structure.
2. Code first
   - The JPA implementation can generate the database schema.
   - Also the key and referencial integrity constrains will be generated programmatically.
   - This approach is frequently use in small projects where the database's sole purpose is to support the application.

On abstraction

- In practice, full abstraction of the database is not desirable:
  1. The DB structure can give guidelines for the application development.
  2. One needs to verify the correct structure of the database, especially in the case where the application builds the schema.
  3. Errors and performance issues force the developer to dive into the DB structure and the communication between the DB and the application.
- Thus, efficient use of JPA requires awareness of the underlying DB structure.

Example: one class, one table



- In the image, there's a class diagram with instance variables.
- id is the identifying variable.
- In addition, there can be other methods and constructors.
  - Typically at least 'getters' and 'setters'.

Example: Employee class

Employee.java

```java
package entity;

import jakarta.persistence.*;

@Entity
public class Employee {

    @Id
    private int id;
    private String firstName;
    private String lastName;
    private double salary;

    public Employee() {
    }

    public Employee(int id, String firstName, String lastName, double salary) {
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
        this.salary = salary;
    }

    @Override
    public String toString() {
        return "Employee [id=" + id + ", firstName=" + firstName + ", lastName=" + lastName +
                ", salary=" + salary + "]";
    }
}
```

Annotation modifiers

- Many of the annotations can be further modified.
- The table and column names can be provided by `name` attribute.
- In addition, there are binary modifiers:
  - `nullable`: can the variable value be empty (the `nullable=false` modifier corresponds to a NOT NULL constraint in the DB)?
  - `insertable`: is it allowed to insert a value for the variable in a new entity?
  - `updatable`: is it allowed to modify the value (it may be undesirable to allow modification of e.g. the primary key)?
- The binary modifiers make Hibernate keep them in force at application level.
  - If not specified, the constraint violation would be detected only at the DB level, in the form of an error message.
- To get placeholders for table and column-specific modifiers, one can insert the following annotations:
  - `@Table`
  - `@Column`

Annotation modifiers

Employee.java

```
@Entity
@Table (name="STAFFMEMBER")
public class Employee {

    @Id
    @Column(name="empnumber", nullable=false, updatable=false)
    private int id;
    @Column(name="fname", nullable=false)
    private String firstName;
    @Column(name="lname", nullable=false)
    private String lastName;
    @Column(name="salary")
    private double salary;
```

#	Name	Datatype	Length/Set	Unsign...	Allow N...	Zerofill	Default
1	**empnumber**	**INT**	**11**	☐	☐	☐	No default
2	fname	VARCHAR	255	☐	☐	☐	No default
3	lname	VARCHAR	255	☐	☐	☐	No default
4	salary	DOUBLE		☐	☑	☐	NULL

company.staffmember: Next Show a

empnumber	fname	lname	salary
1	Mary	Smith	3 600

Auto-generated keys

Employee.java

```
@Id
@GeneratedValue(strategy=GenerationType.AUTO)
private int id;
```

- @GeneratedValue annotation tells the DBMS to generate the primary key values automatically.
  - The strategy attribute defines how the values are generated and stored.
    - GenerationType.AUTO leaves the decision up to the DBMS.
- The instance variable that contains the value must be left unspecified at the application level (to the default value of 0).
- Hibernate automatically runs a SELECT statement after UPDATE, querying for the newly-obtained value.
  - Thus, the value is immediately usable in the application after the object is persisted.

Date and timestamp fields

- The date and timestamp fields are a constant source of frustration in database applications.
  - DBMS's provide non-standard data types.
  - Time zones may cause problems.
- JPA supports these Java classes:
  - Java.util.Date
  - Java.util.Calendar
- At database level, JPA supports three SQL temporal types:
  - DATE
  - TIME
  - TIMESTAMP
- In addition, JPA 2.2 provides support for Java Date and Time API.

@Temporal

Employee.java

```
@Entity
public class Employee {

    @Id
    private int id;
    private String firstName;
    private String lastName;
    private double salary;
    @Temporal(TemporalType.DATE)
    private java.util.Date savedDate;
```

- The database-level implemetation is selected by @Temporal annotation.
- In the example, the application-given timestamp is saved into a DATE field in the database.

company.employee: 1 Next Show all

id	firstName	lastName	salary	savedDate
1	Mary	Smith	3 600	2019-03-27